组内技术分享 · 2026 年 10 月

Agent Trace
记录、评估与记忆

trace 的概念与用途，以及组内在摄入与存储、失败归因、ARC-AGI-3 / VISTA 上的工作

Agent Trace 组 · 曹金羽

基础概念

Chat、Trace 与 Memory

Chat 记录

线性文本 · 按会话

记录什么

系统提示、用户消息、模型回复、工具结果，即模型实际看到的上下文

用途

继续对话、人工翻阅；过长时被压缩或截断

Trace

结构化执行记录 · 按运行

记录什么

每次 LLM 调用与工具调用的起止时间、父子关系、状态、模型与 token、输入输出

用途

可观测性、评估、失败归因、回放与续跑

Memory

提炼后的条目 · 跨任务

记录什么

从历史执行中提炼、持久保存、在后续任务中注入上下文的经验

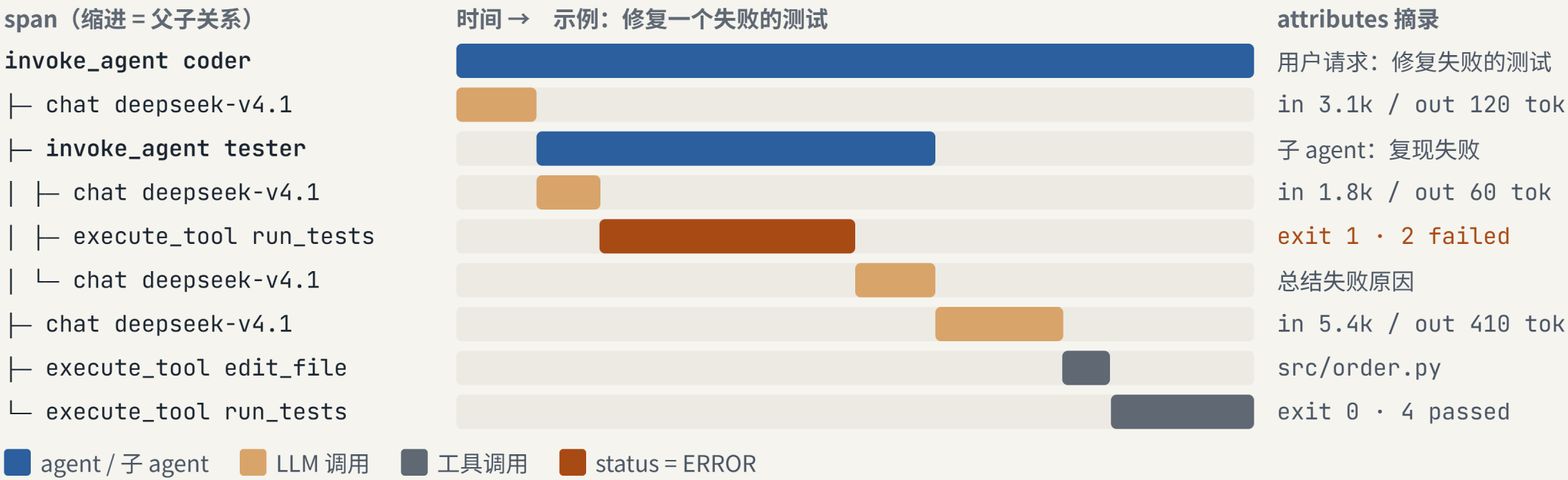
用途

避免重复错误、复用已验证的做法、遵循用户偏好

chat 是 trace 中 LLM 调用的输入输出部分；现有记忆生成工作大多先把 trace 转为纯文本，父子结构、耗时和状态码随之丢失。

基础概念

Trace 的结构



每行是一个 span：parent_span_id 指向上一层，整棵树共享一个 trace_id。span 是主要的写入与分析单元，trace_id 是关联键。

基础概念

Trace 的用途

可观测性

总延迟由哪些步骤组成；各模型消耗多少 token 与成本

评估

某个模型、agent 版本或工具在一段时间内的成功率与延迟分布；离线评测逐步打分

失败归因

失败发生在哪个模型或工具调用；哪个 agent、哪一步引入了决定性错误

记忆与回放

从历史 trace 生成记忆供后续任务使用；按步取回当时的状态，断点续跑

单次运行的问题按 trace_id 重建一棵树；统计类问题跨大量 span 筛选与聚合。负载形态是持续追加写入加时间范围分析（append-heavy OLAP）。

组内工作

OpenTelemetry：我们用到的部分

OpenTelemetry（OTel）是 CNCF 的可观测性开放标准，统一规定 trace、metric、log 的数据模型、SDK 与传输协议 OTLP。它最初用于微服务的分布式追踪，即记录一个请求跨多个服务的调用链；GenAI 语义约定把同一套模型扩展到 LLM 与 agent 调用。

agent / 数据生产端

OTel SDK 埋点；trace-synthesis 合成回放

OTLP/HTTP →

Collector

Receiver → Batch → 自研 openGauss Exporter

SQL 写入 →

存储

openGauss 行存 / xstore 列存 / PostgreSQL

数据模型

- Resource → Scope → Span 三层
- trace_id / span_id / parent_span_id
- attributes、events、status

GenAI 语义约定

- `gen_ai.request.model`
- `gen_ai.usage.*_tokens`
- `gen_ai.input.messages` / `output.messages`

Collector 定制

- OCB 构建含自研 exporter 的发行版
- 发送队列：重试、背压、并行写入
- accepted / sent 指标用于对账

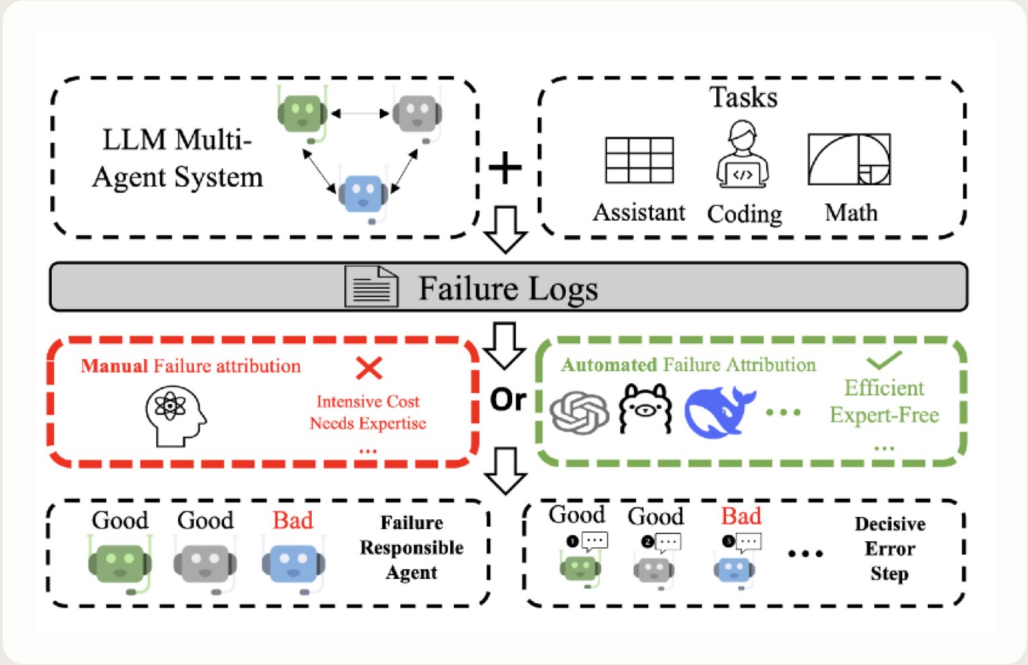
组内工作

存储侧实验

实验	目的	工作	结果
摄入对账	trace 从产生到入库不丢失、不重复	经 Collector 写入 openGauss，发送清单与库内数据做差集	10 traces / 69 spans 全部入库，差集为 0
列存写入	持续追加写入的吞吐与完整性	50 万 spans 写入 xstore 列存，exporter 多路并行	无丢失；8 路并行 10,316 rows/s
查询去重	OTLP 重试会产生重复 span，查询须得到唯一结果	只追加、不建主键，查询时按 (trace_id, span_id) 取最新版本	先过滤再去重：约 4 s 降至 141–257 ms
图查询	按调用关系取整棵 trace 树，做跨 trace 关联	span 父子关系存为图的边，用 Cypher 查询，与列存对比	取整树快 23 倍；取 payload 慢 1.5 倍，图只做关系查询层
payload 存储	prompt / response 等长文本占主要体积	对比同表、拆表、外置等布局；分析 3,600 条真实轨迹	每次 LLM 调用记录完整上下文时，存储量是只记录增量的 53.15 倍

来源：exporter_demo、agent-trace-graph-test、agent-trace-research 仓库实验报告

失败归因：Who&When



图：Who&When 论文任务示意（arXiv:2505.00212）

给定一条失败的多 agent 执行轨迹，找出**责任 agent**（who）和引入**决定性错误的步骤**（when）。

Who&When（ICML 2025）：184 条人工标注失败轨迹，任务来自 GAIA 与 AssistantBench

69.05%
agent 级准确率

21.43%
步骤级准确率

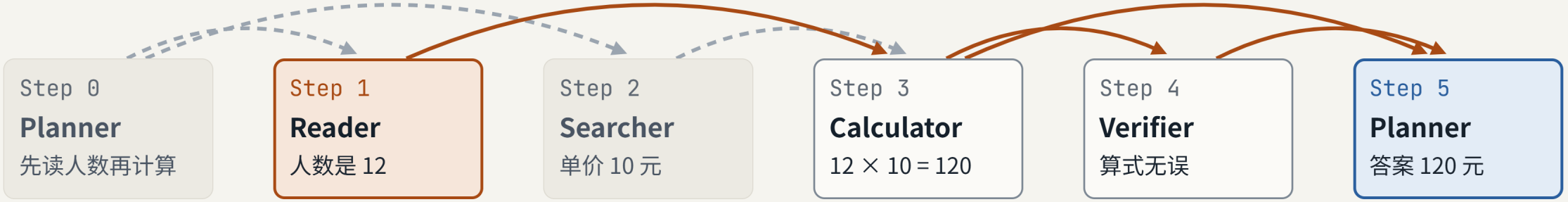
All-at-Once（整条轨迹交给 LLM 一次判断），DeepSeek-V4.1-Flash，算法生成子集 126 例。人工构造子集轨迹更长，步骤级准确率为 5.17%。

来源：github.com/mototale/Agents_Failure_Attribution 基线复跑

失败归因

DAG 回溯：流程示例

示例（虚构）：多 agent 系统计算“每人 10 元、10 人的总额”，最终输出 120 元，正确答案是 100 元。



起点：失败表现　已检查：本步无错误　候选：本步引入错误　未回溯　实线：回溯经过的依赖边

1 构图

LLM 读完整轨迹，标出每一步用到了哪些前序步骤的结果

2 确定起点

找到失败表现所在步：Step 5 答出 120 元，与正确答案不符

3 回溯

沿依赖边反向遍历：5、4、3 本身无错误；Step 1 读错人数，记为候选

4 裁决

只有一个候选，输出 Step 1；agent 由该步推出：Reader

来源：github.com/mototale/Agents_Failure_Attribution，docs/dag_backtrace_design.md

失败归因

DAG 回溯：结果

All-at-Once（基线）

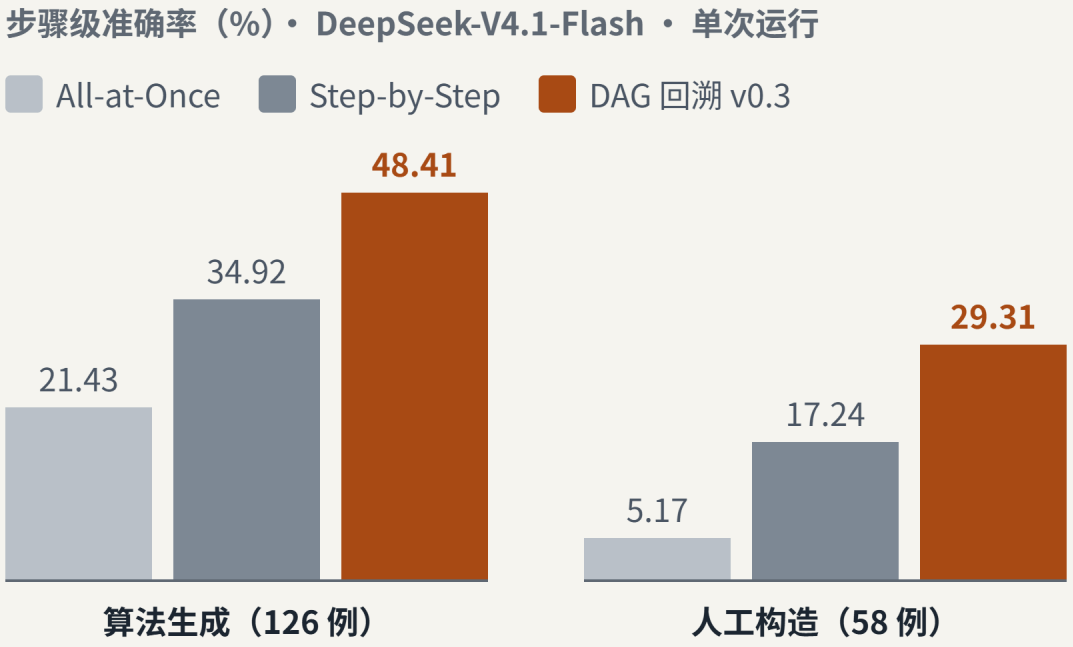
整条轨迹一次交给 LLM，直接回答哪个 agent、哪一步

Step-by-Step（基线）

从第一步开始逐步判断“这一步是否出错”，在第一个判为出错的步骤处停止

DAG 回溯

从失败点出发，只沿依赖边检查上游，不检查与失败无关的分支

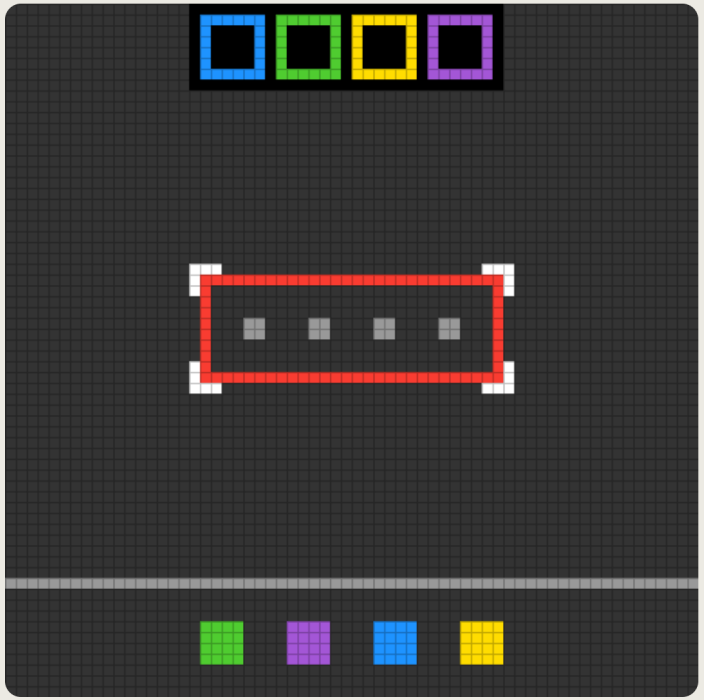


token 消耗为基线的 2.3–4.8 倍；每个步骤级错误可归入“构图不可达 / 未回溯到 / 局部误判 / 裁决选错”四类之一，便于逐环节改进。

来源：github.com/mototale/Agents_Failure_Attribution，experiments/v0.3

ARC-AGI-3 × VISTA

ARC-AGI-3：没有规则的交互式游戏



sb26 第 0 回合：模型得到的全部输入

试玩：arcprize.org/tasks/ls20

全部公开游戏 · ARC-AGI-3 官网

环境

回合制抽象游戏，64×64 网格、16 色；只给画面和动作，不给规则与目标

要求

探索、推断胜利条件，并迁移到后续关卡

计分 RHAE

每关（人类动作数 ÷ AI 动作数）^{2}，只计改变游戏状态的动作；人类基线取首次接触的玩家

规模

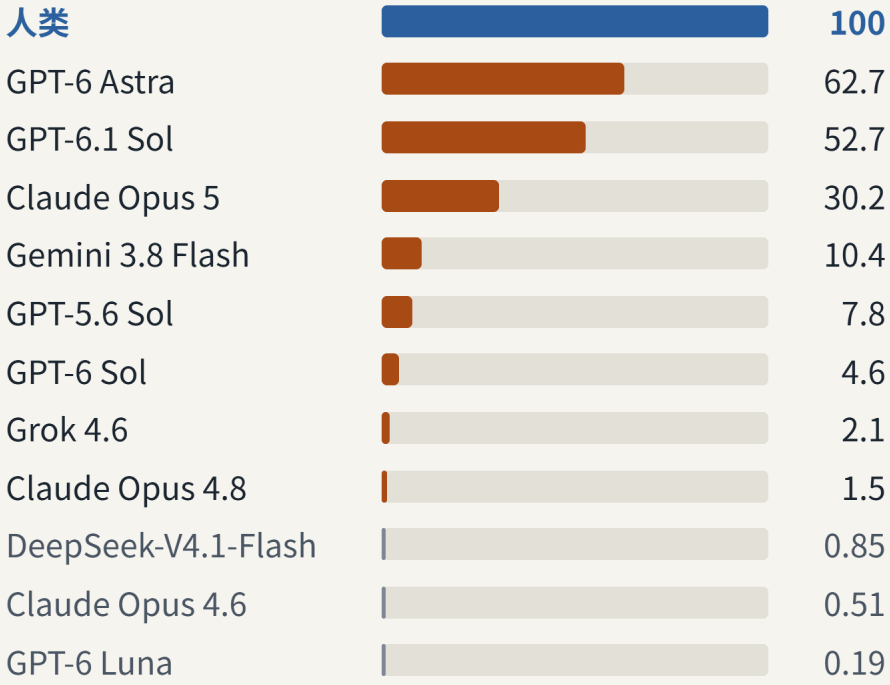
公开集 25 个游戏、183 关；人类可解全部环境

进展

2026 年 3 月发布时前沿模型均低于 1%；半年后最高 62.7%，多数模型仍在 1% 以下

官方榜单 RHAE（%）

Standard harness · 无工具 · 截至 2026-10-14

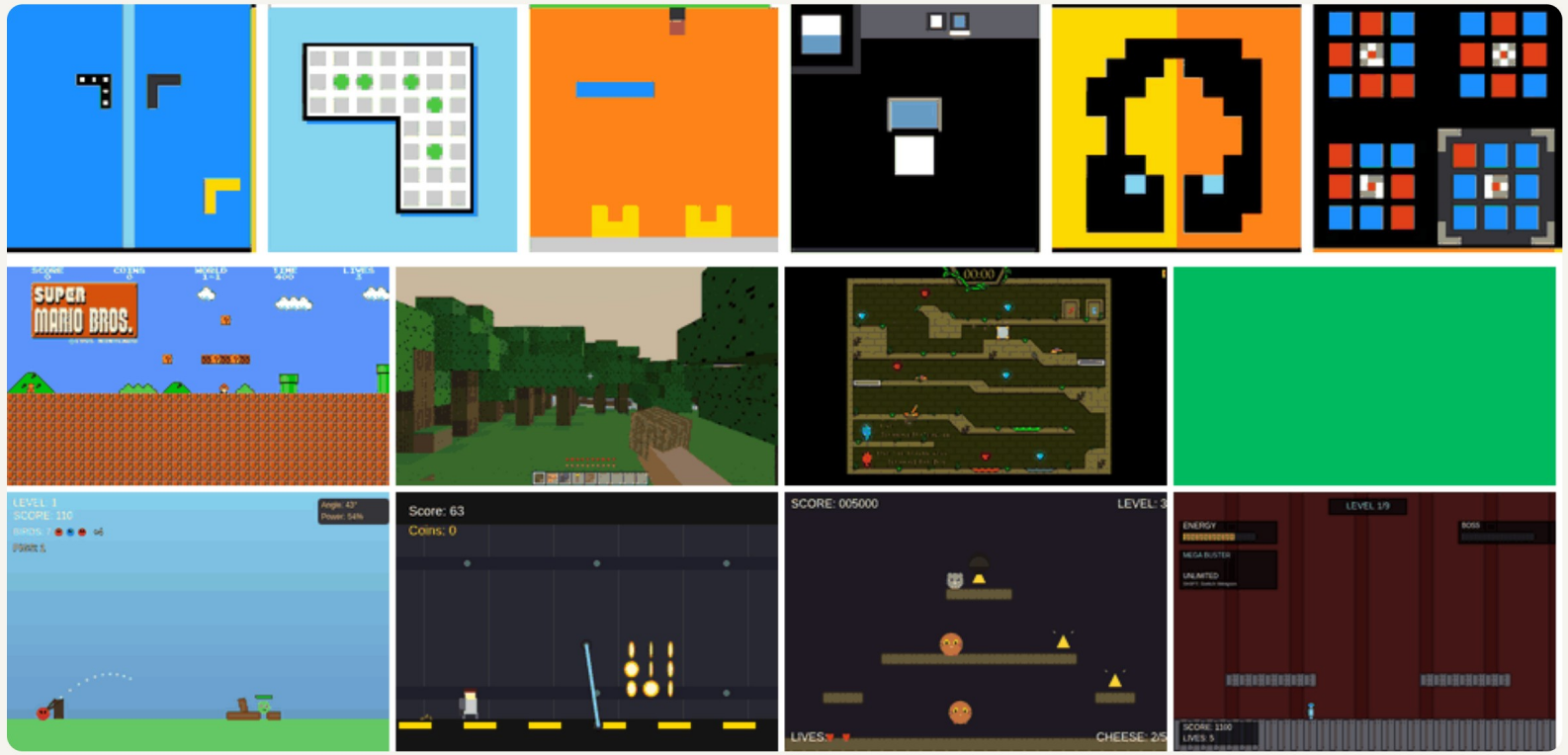


完整榜单：arcprize.org/leaderboard

来源：arXiv:2603.24621；榜单分数经 llmlearner.com、benchlm.ai 汇总页核对，以官网为准

ARC-AGI-3 × VISTA

VISTA：无损视觉记忆



- 每帧画面完整保存为图片，整局保留
- inspect / read_pixels / history：按回合号取回任意画面、像素与动作记录
- GUIDE.md 记录跨关理解，WORKING.md 记录本关草稿
- 系统提示仅 9 行，不给规则与策略，模型不能执行代码

作者结论：模型能力已足够，限制因素是缺少可随时查阅的无损记录。

40.68 → 100
Opus 5.0 RHAE：最小接口 → VISTA

183 / 183
公开集 25 个游戏全部关卡通过

7,542
总动作数；首次游玩的人类玩家为 17,135

来源：Han et al.（MIT，Kaiming He 组），arXiv:2610.02200 · vista-research.github.io

VISTA 的 8 个工具与系统提示

工具	作用	输入 → 返回
play	执行一个动作	ACTION1–7 或 RESET，点击另带坐标 → 元数据 + 本回合最后一帧
inspect	回看画面	1–16 个视图（回合号、帧号、区域）→ 放大后的截图
read_pixels	测量颜色	1–64 个视图，按行列网格采样 → 颜色符号矩阵
history	查动作记录	events：逐回合动作与结果；attempts：本关各次尝试 → 文字
read_guide write_guide	长期笔记	全文读写，至多 64 KB，整局保留
read_working write_working	本关草稿	全文读写，至多 16 KB，过关时归档清空

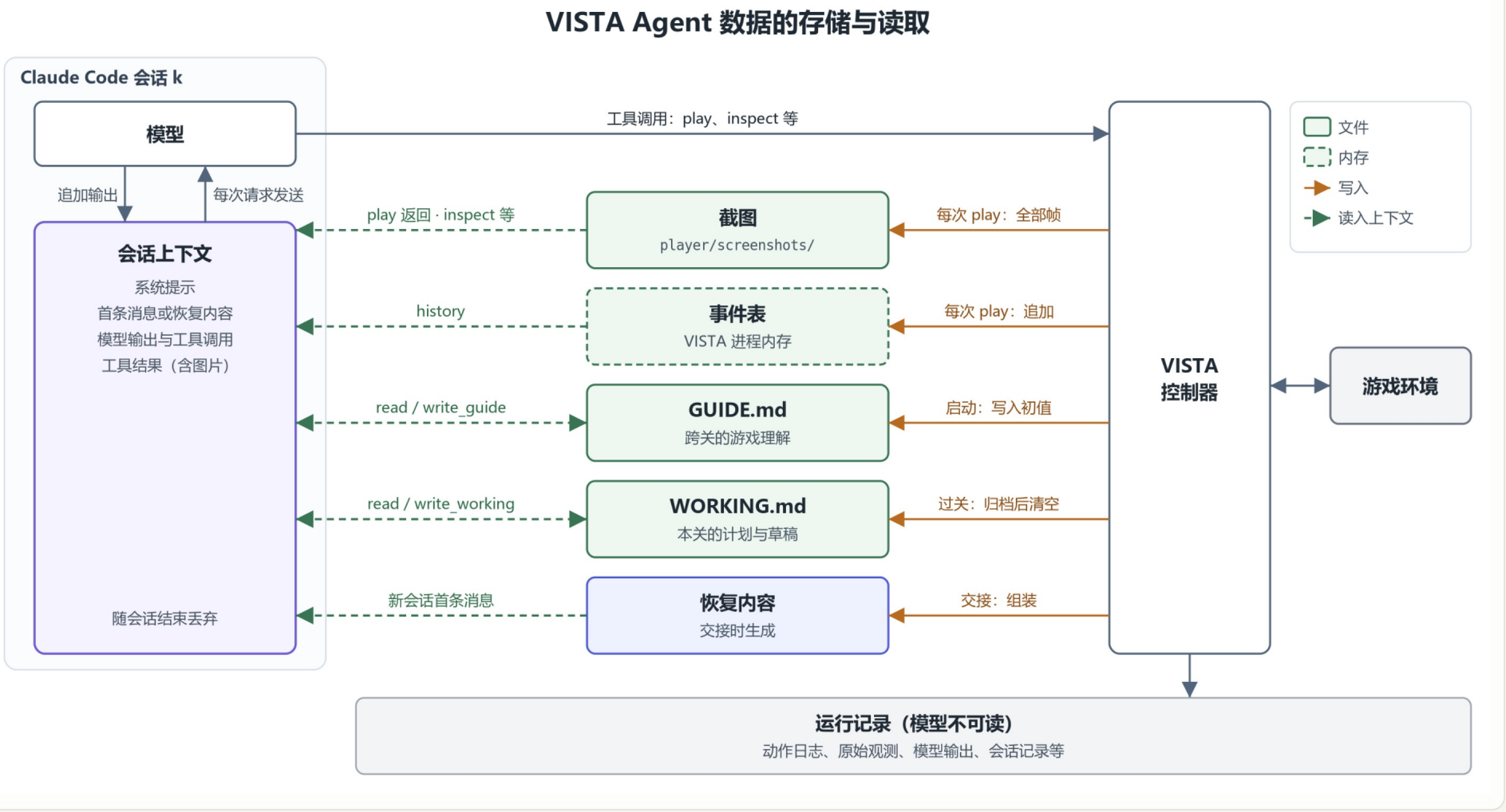
系统提示（全文）

Visual game task. Complete the game with as few game actions as possible. Build and use a compact, revisable model of the game and its current state. Update it as new evidence changes what is supported. Before each play, briefly state what you expect to see. Afterward, briefly state all visible changes, expected or not. Keep concise, durable, revisable game understanding in GUIDE.md; use WORKING.md as a scratchpad when useful.

- 只有 play 自动返回画面；更早的画面、动画帧和动作记录需要模型按回合号主动查询
- 内置工具全部禁用，模型不能读文件、不能执行代码

来源：agent-trace-research/docs/arcagi/vista-runtime-data-flow.md §3；github.com/joshhhhhan/VISTA

VISTA 的无损游戏记录



随会话丢弃

左侧会话上下文，上下文压缩时整体丢弃

整局保留

中间的截图、事件与笔记，模型按回合号随时取回

广义 trace

按回合平铺，只含游戏信息，读取方是运行中的 agent

图：agent-trace-research/docs/arcagi，VISTA 运行时数据流

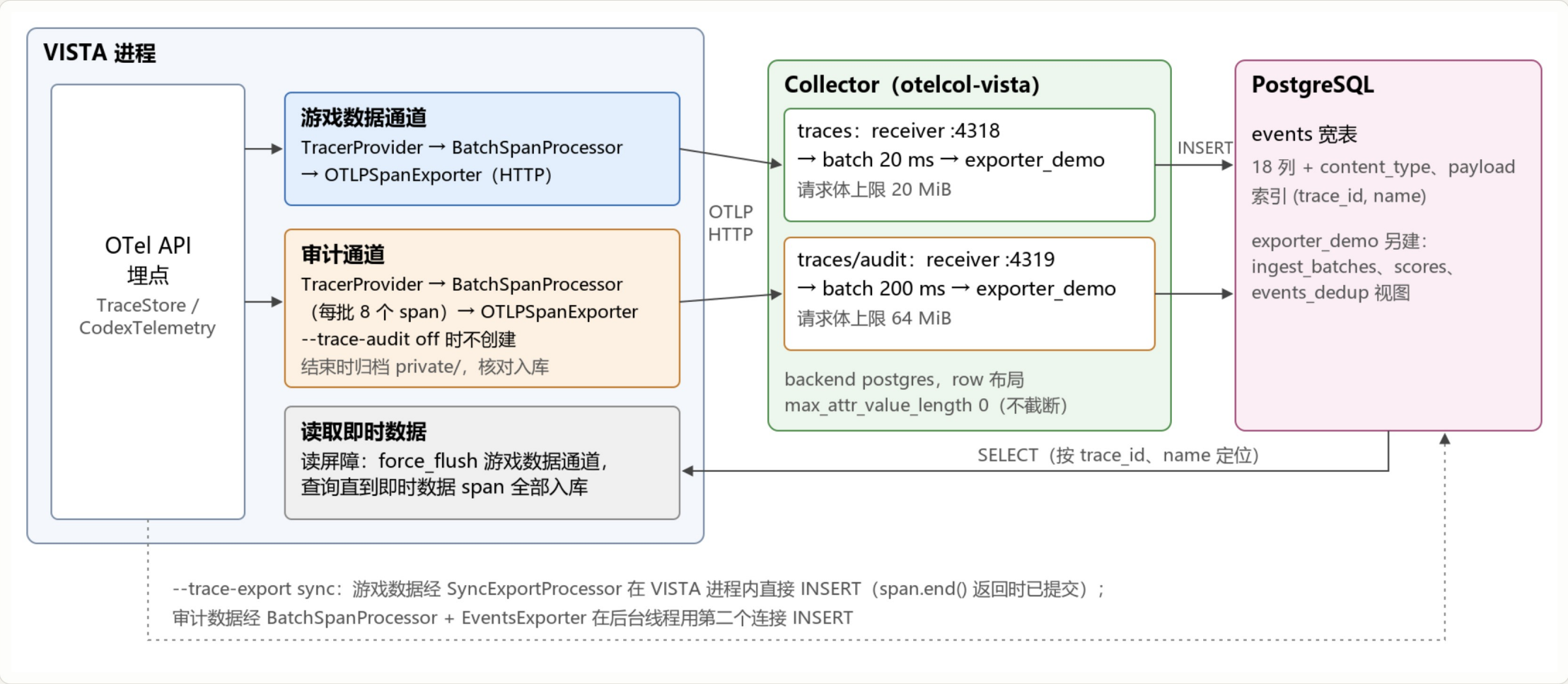
我们的工作

两层记录：游戏即时数据与执行全量记录

	游戏即时数据（VISTA 核心）	执行全量记录（我们的 trace）
内容	截图、动作、GUIDE / WORKING	另含 LLM 调用、工具调用、会话段、上下文压缩、ARC 请求、private/ 原文
形态	文件系统，按回合平铺	OTel span 树，结构化写入数据库
读取方	运行中的 agent	开发者，以及评估与优化流程
用途	无损记忆：随时回看任意回合	评估、失败归因、断点续跑、改进 harness、论文分析

我们的工作

执行全量记录的 OTel 实现



一局 = 一个 trace；读取即时数据前经读屏障等待入库；500 步回放与文件存储逐字节一致，每次工具调用额外约 7 ms。

来源：vista-trace-collector/docs/trace-storage-design.md（trace-v5.1）

小结

Agent Trace 的关键点

记录

trace 是一次运行的结构化操作树，包含 chat 记录和 memory 中没有的结构、时间与状态信息

存储

OTel 统一数据模型与传输；存储侧是追加写入加时间范围分析，需处理去重和长 payload

归因

步骤级定位明显难于 agent 级；利用执行依赖结构可以提高步骤级准确率

无损记录

可随时查阅的无损记录直接影响 agent 的任务表现；完整的结构化记录是评估与优化的依据

Q & A

参考链接

ARC-AGI-3	arcprize.org/arc-agi/3 · arXiv:2603.24621
VISTA	arXiv:2610.02200 · vista-research.github.io · github.com/joshhhhhan/VISTA
Who&When	arXiv:2505.00212 · github.com/mototale/Agents_Failure_Attribution
OTel GenAI 语义约定	github.com/open-telemetry/semantic-conventions-genai
Langfuse	github.com/langfuse/langfuse